Компьютерные науки и анализ данных

Yandex

Faculty of Computer science
Higher School of Economics

Программирование на Python

лектор: Сергей Горшков

План

- Субгенераторы
- Асинхронность
- Asyncio

Обработка ошибок

- ***Исключения — не «ошибки», а способ обработки некоторых условий не там, где они были обнаружены (Г.В.Курячий).***
- Сообщение об ошибке должно быть информативным
- Выбрасывается ошибка с помощью команды raise: `raise TypeError("msg")`
- Ловится и обрабатывается с помощью [try-except [..else-finally]]

```
try:
    k = 1 / 0  # в блоке try может вылететь ошибка
except ZeroDivisionError as e:
    print("Ошибка при делении на ноль", e)  # обработка ошибки
else:
    print("Всё хорошо, ошибок не было")
finally:
    print("Это вызовется в любом случае")
```

Создание своего класса ошибки

- Для выбрасывания ошибок в каком-то классе/модуле можем сконструировать специальный класс, содержащий подробное описание ошибки и все необходимые поля и выбросить его в то место, которое способно его обработать
- Класс для ошибки должен быть наследником BaseException. Соответственно, для заполнения полей надо переопределить метод __init__. Если хотим, чтобы печаталась строка при обработке

```
except MyException as e:
    print(e)
```

  надо переопределить метод __str__ (а мы знаем, что можно и __repr__)
- Также можно обращаться к определенным полям экземпляра `e`

Асинхронность и параллелизм

- Параллелизм – есть параллельные процессы, каждый в своем ритме делает работу и они друг с другом периодически обмениваются сообщениями, т.к. они все делают одну и ту же работу параллельно (это как раз синхронность!!!)
- Параллелизма в синтаксисе ЯП Python нет
- Виджеты должны работать асинхронно – нажал пользователь на кнопку, и запускается обработчик этого нажатия. Программисты как бы пишут цикл, который будет эти события отсматривать, в виджетах будут коллбэки, которые мы будем дёргать
- В этой модели нет ничего, что можно было бы добавить в синтаксис языка

Синтаксическая асинхронность

- Возможность выполнения кода в различных местах – внутри кода разметите сегменты и определите правила для их запуска. Это корутины, они же сопрограммы
- Сопрограмма – функция, в которую можно много раз входить (входим не в начало, а в то место, из которого вышел) и, возможно, при входе будут передаваться дополнительные данные.
- Асинхронность сопрограмм – когда из неё частично вышли, код сопрограммы перестает выполняться, и выполняется другой код. Когда мы вернулись в эту сопрограмму, он продолжает выполняться с того же места, где остановились

yield from

- Напомним, генератор это объект который можно проитерировать только однажды.
- Пусть есть два генератора и надо проитерировать один генератор, потом второй. Но вернуть значения не в виде списка, в виде генератора.
- Конструкция yield from позволяет «вкладывать» один генератор в другой, то есть создавать субгенераторы.
- yield from позволяет программисту легко управлять сразу несколькими генераторами, настраивать их взаимодействие и, конечно, заменить более длинную конструкцию for+yield

yield from

- В обычном yield return просто выбрасывал исключение StopIteration
- Можно было получить возвращаемое значение, обработав исключение, получив поле value

- Если субгенератор возвращает значение с помощью return и вызывается с помощью yield from, то инструкция yield from будет выдавать возвращаемое значение
- Создадим параметрический итератор, который будет принимать значения через send. Обратите внимание, что значения, получаемые основным генератором, будут пробрасываться в субгенераторы в случае использования yield from

Асинхронность руками: подготовка

- Фрагменты кода, разделенные yield-ами, в явно асинхронном порядке, который определяет цикл, выполняется последовательно, но по кусочкам

```
def sub(name):
    yield f"{name} >"
    yield f"{name} <"
def gen(name):
    for i in range(100000):
        yield from sub(name)
        yield f"{name}-{i}"
cores = gen("1st"), gen("2nd")
for i in range(100):
    sleep(0.1)
    res = next(cores[i % 2])  # random()>0.5, 0.75
    print(f"({i:02}) {res}")
```

Код

Переходим к async

- async def ~== генератор
- yield from ~== await
- @types.coroutine — чтобы и yield и return (не доделали ещё?)
- async def + yield — совсем другое, это именно то, чем кажется, специальные асинхронные генераторы, которые можно проходить async for (причём в конструкторах вида [… async for i in асинхронный-гененратор …] тоже)
- наш sub() — не совсем стандартная корутина, потому что использует прямое управление образующим циклом с помощью yield
- В готовом инструментарии это практически никогда не нужно: и образующий цикл, и инструменты управления им должны входить в такой инструментарий

Asyncio

- Хочется писать асинхронные программы, лишь создавая async def .. await, а всё остальное – обмазка
- Самое сложное и самое ненужное – образующий цикл
- В main loop можно забрасывать future
- Future – асинхронная функция, в которой есть только 1 yield – отдаем ее коду, который ждет на ней await (ждет, когда это future сбудется) – ставимся в main loop
- Этот код спит на ней
- Другой код заполняет результат и выставляет готовность
- mainloop проверяет готовность и передает управление по готовности

Asyncio, create_task

- asyncio.run(main()) — запуск «приложения» main() в образующем цикле asyncio()
- «приложение» asyncio — корутина, который заполняет очередь mainloop-а и немножко командует им
- Если просто написать await — корутина «просто запустится», в чём асинхронность, непонятно (даже если она и выходила в mainloop)
- В примере первая корутина спит секунду, а вторая — две
- Если написать create_task(корутина), то её выполнение тут же стартует в mainloop-е
- В примере ещё две корутины планируются одновременно, первая из них спит три секунды, а вторая — четыре, так что отрабатывает через секунду после первой
- asyncio.sleep(тайм-аут) — это команда mainloop-у «верни мне управление после тайм-аута»

Asyncio, Gather

- Функция gather() модуля asyncio одновременно запускает объекты *awaitable*, переданные в функцию как последовательность *aws.
- Функция asyncio.gather() представляет то же объект ожидания awaitable и запускается оператором *await*.
- Если какой-либо объект *awaitable* в последовательности *aws является сопрограммой, то она автоматически назначается как задача asyncio.Task.

```
async with asyncio.TaskGroup() as tg:
    task1 = tg.create_task(some_coro(...))
    task2 = tg.create_task(another_coro(...))
print(f"Both tasks have completed now: {task1.result()}, {task2.result()}")
```

Asyncio, Future

- **Future** — awaitable-объект, у которого есть творец и адресат, оба имеют к нему доступ.

1. Адресат делает результат = await фьюча, и уходит в mainloop.
2. Творец, когда ему заблагорассудится, с помощью фьюча. set_result(что-то) объявляет mainloop-у, что будущее наступило с результатом что-то
3. Тогда mainloop, наконец, активизирует эту фьючу и адресат просыпается

- В действительности, фьюча — это просто генератор на два шага: будущее не наступило / yield / будущее наступило, в котором есть два дополнительных поля (объявление о том, что оно наступило и передаваемое значение)

Faculty of Computer Science
Higher School of Economics

sgorshkov@hse.ru
tg: @Seragor98